Also, the second query renames some columns with the AS statement so that the web application displays the values from the user table. Be sure to update your Rails to at least 2.1.1.

7.2.4 Countermeasures

Ruby on Rails has a built-in filter for special SQL characters, which will escape ' , " , NULL character and line breaks. *Using `Model.find(id)` or `Model.find_by_some thing(something)` automatically applies this countermeasure*. But in SQL fragments, especially *in conditions fragments (`where("...")`), the `connection.execute()` or `Model.find_by_sql()` methods, it has to be applied manually*.
Instead of passing a string to the conditions option, you can pass an array to sanitize tainted strings like this:

```
Model.where("login = ? AND password = ?", entered_user_name,
entered_password).first
```

As you can see, the first part of the array is an SQL fragment with question marks. The sanitized versions of the variables in the second part of the array replace the question marks. Or you can pass a hash for the same result:

```
Model.where(login: entered_user_name, password:
entered_password).first
```

The array or hash form is only available in model instances. You can try `sanitize_sql()` elsewhere.*Make it a habit to think about the security consequences when using an external string in SQL*.

7.3 Cross-Site Scripting (XSS)

The most widespread, and one of the most devastating security vulnerabilities in web applications is XSS. This malicious attack injects client-side executable code. Rails provides helper methods to fend these attacks off.

7.3.1 Entry Points

An entry point is a vulnerable URL and its parameters where an attacker can start an attack.

The most common entry points are message posts, user comments, and guest books, but project titles, document names and search result pages have also been vulnerable - just about everywhere where the user can input data. But the input does not necessarily have to come from input boxes on web sites, it can be in any URL parameter - obvious, hidden or internal. Remember that the user may intercept any traffic. Applications, such as the Live HTTP Headers Firefox plugin, or client-site proxies make it easy to change requests.
XSS attacks work like this: An attacker injects some code, the web application saves it and displays it on a page, later presented to a victim. Most XSS examples simply display an alert box, but it is more powerful than that. XSS can steal the cookie, hijack the session, redirect the victim to a fake website, display advertisements for the benefit of the attacker, change elements on the web site to get confidential information or install malicious software through security holes in the web browser.

During the second half of 2007, there were 88 vulnerabilities reported in Mozilla browsers, 22 in Safari, 18 in IE, and 12 in Opera. The Symantec Global Internet Security threat report also documented 239 browser plug-in vulnerabilities in the last six months of 2007. Mpack is a very active and up-to-date attack framework which exploits these vulnerabilities. For criminal hackers, it is very attractive to exploit an SQL-Injection vulnerability in a web application framework and insert malicious code in every textual table